

A Mathlib-Idiomatic Formalization of the Fricke Involution and its Operator on Modular Forms

Xavier Callens*

SocrateAI Research Initiative

September 6, 2026

Archived at Zenodo: [10.5281/zenodo.22541930](https://zenodo.org/record/22541930)

Abstract

We formalize, in Lean 4 on top of Mathlib, the Fricke involution $W_N = \begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}$ on $\Gamma_0(N)$ and the operator it induces on modular forms, as a small reusable API rather than as proof-internal infrastructure. We claim no priority: Fricke and Atkin–Lehner material already exists in Lean 4 in the Fermat’s Last Theorem artifact [8] (§6), which covers strictly more. What we offer is a packaging aimed at upstreaming — Mathlib itself contains no Fricke or Atkin–Lehner operator. Mathlib already provides the upper half-plane, the Möbius action of $GL_2(\mathbb{R})$, the weight- k slash action carrying the determinant factor, the congruence subgroups $\Gamma_0(N)$, and the bundled types `SlashInvariantForm` and `ModularForm`; it contains no Fricke or Atkin–Lehner operator. We supply that missing piece in three layers: (i) the matrix level — $W_N \in GL_2^+(\mathbb{R})$ with $\det W_N = N$, the involution identity $W_N^2 = -N \cdot I$, the explicit conjugation $W_N \gamma W_N^{-1} = \begin{pmatrix} d & -c/N \\ -Nb & a \end{pmatrix}$, and the theorem that W_N normalizes $\Gamma_0(N)$; (ii) the slash level — $f \mapsto f|_k W_N$ preserves $\Gamma_0(N)$ -slash-invariance; and (iii) the bundled level — the operator is well defined on `ModularForm`, its holomorphy and boundedness at the cusps transported along W_N . The development is 27 declarations in 291 lines; all 21 theorems compile with no `sorry` and depend on exactly Lean’s three standard axioms `propext`, `Classical.choice` and `Quot.sound`, pinned by 16 build-failing `#guard_msgs` checks so that a drifted footprint fails compilation. We make no claim of new mathematics: the results are classical, due to Fricke and to Atkin–Lehner [1]. The contribution is the mechanization, and its intended use is as infrastructure for formalizing eta-quotient identities, where Fricke eigenvalues are the organizing invariant.

MSC 2020: 11F11, 11F06, 68V20. **Keywords:** Fricke involution, Atkin–Lehner theory, congruence subgroups, Lean 4, Mathlib, formal verification.

1 Introduction

The Fricke involution is elementary to state and ubiquitous in the theory of modular forms on $\Gamma_0(N)$. For $N \geq 1$ set

$$W_N = \begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}, \quad \det W_N = N > 0.$$

Acting on the upper half-plane by $\tau \mapsto -1/(N\tau)$, it exchanges the cusps 0 and ∞ of $\Gamma_0(N)$, normalizes $\Gamma_0(N)$, and induces an involution on $M_k(\Gamma_0(N))$ whose ± 1 eigenspaces organize much of the arithmetic of the space — most visibly for eta-quotients, where multiplicativity is read off from Fricke and Atkin–Lehner eigenvalues [7, 6].

*Corresponding author: callensxavier@gmail.com

This note is written in the mode that recent practice makes standard: a formal artifact shipped alongside the human exposition. Large-scale autoformalization has crossed a threshold — a complete machine-checked proof of Fermat’s Last Theorem was produced in eleven days by AI agents coordinating through a shared statement graph [8, 9] — and the surrounding methodological questions are now mainstream mathematics [10, 3]. We work in Lean 4 [12] against Mathlib [2], and we adopt two specific practices from the FLT artifact: the axiom audit as a *build target*, and an explicit statement-dependency graph (§5).

What Mathlib already provides

We are explicit about prior art, because it determines what is and is not new here. As of commit 905b9581 (2026-07-28), Mathlib [2] contains, in forms strictly more general than we would need to restate:

- `UpperHalfPlane` and the Möbius action of $GL_2(\mathbb{R})$, together with `denom_ne_zero_of_im` and `normSq_denom_pos`;
- `im_smul_eq_div_normSq`, which is exactly the classical identity $\text{Im}(M \cdot \tau) = |\det M| \text{Im}(\tau) / |c\tau + d|^2$, stated for arbitrary $M \in GL_2(\mathbb{R})$ via $|\det M|$ rather than only for $\det M > 0$;
- `Matrix.GLPoS`, the subgroup of positive determinant;
- `CongruenceSubgroup.Gamma0 N`;
- the weight- k slash action `SlashAction ℤ (GL (Fin 2) ℝ) (ℍ → ℂ)`, whose definition *already carries* the determinant factor, $f|_k g = \sigma(g) f(g \cdot \tau) |\det g|^{k-1} \text{denom}(g, \tau)^{-k}$;
- the bundled `ModularForm Γ k` for $\Gamma : \text{Subgroup } (GL_2(\mathbb{R}))$, with holomorphy and boundedness at cusps.

None of that is reproved here; we import it. A reader looking for a formalization of the $GL_2^+(\mathbb{R})$ -action on $\mathbb{H}$ should use Mathlib’s, not ours.

The gap, and what we contribute

A repository-wide search of Mathlib at that commit for `Fricke` or `AtkinLehner` returns *zero* occurrences: the machinery to host these operators exists in Mathlib, but the operators do not. That is the gap we address — with the important qualification, developed in §6, that Fricke material *does* already exist in Lean 4 outside Mathlib. Our contributions are therefore stated as an API, not as firsts:

- (C1) W_N as an element of $GL_2(\mathbb{R})$ with $\det W_N = N$, and $W_N \in GL_2^+(\mathbb{R})$ (Theorem 3.1);
- (C2) $W_N^2 = -N \cdot I$ (Theorem 3.2), which since scalar matrices act trivially on $\mathbb{H}$ is the involution property;
- (C3) the explicit conjugation formula (Theorem 3.3);
- (C4) W_N conjugates $\Gamma_0(N)$ into itself inside $GL_2(\mathbb{R})$ (Theorem 3.4); together with $W_N^2 = -N \cdot I$ central this gives the two-sided normalizer condition, though we formalize the containment;
- (C5) slashing by W_N preserves $\Gamma_0(N)$ -slash-invariance, giving the operator on `SlashInvariantForm` (Theorem 4.1);
- (C6) holomorphy and boundedness at the cusps transport along W_N , giving the operator on the bundled `ModularForm` (Theorem 4.2);

(C7) the composite $f \mapsto (f|_k W_N)|_k W_N$ is multiplication by the explicit scalar $(-1)^k N^{k-2}$ (Theorem 4.3), which identifies the normalization making $|_k W_N$ an involution.

(C4) is the hinge: it is what makes $f \mapsto f|_k W_N$ well defined at all. (C5) and (C6) carry it up to Mathlib’s bundled types, so that the Fricke operator can be composed with the rest of the library rather than living beside it.

2 Mathematical background

Let $\gamma = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z})$ with $\gamma \in \Gamma_0(N)$, i.e. $N \mid c$; write $c = Nc'$. A direct computation gives

$$W_N \gamma W_N^{-1} = \begin{pmatrix} d & -c' \\ -Nb & a \end{pmatrix}, \quad (1)$$

whose entries are integers precisely because $N \mid c$, whose determinant is $da - (-c')(-Nb) = ad - bc = 1$, and whose lower-left entry $-Nb$ is divisible by N . Hence the conjugate again lies in $\Gamma_0(N)$, and W_N normalizes $\Gamma_0(N)$.

Rather than manipulate W_N^{-1} in the formalization, we prove the equivalent *intertwining* identity

$$W_N \cdot \gamma = \delta \cdot W_N, \quad \delta := \begin{pmatrix} d & -c' \\ -Nb & a \end{pmatrix}, \quad (2)$$

which avoids inverses entirely and from which (1) follows by one right multiplication. Both sides of (2) equal $\begin{pmatrix} -c & -d \\ Na & Nb \end{pmatrix}$, the two agreeing exactly on the relation $c = Nc'$.

3 The Lean development

The development is four modules. We reproduce the load-bearing declarations verbatim from the compiled source.

```
def frickeMatrix (N : ℕ) : Matrix (Fin 2) (Fin 2) ℝ := !![0, -1; (N : ℝ), 0]

@[simp] theorem frickeMatrix_det (N : ℕ) : (frickeMatrix N).det = (N : ℝ) := by
  simp [frickeMatrix, Matrix.det_fin_two_of]

noncomputable def frickeW {N : ℕ} (hN : 0 < N) : GL (Fin 2) ℝ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero (frickeMatrix N) (frickeMatrix_det_ne_zero hN)
```

Listing 1: The Fricke matrix and its determinant.

Theorem 3.1 (W_N has positive determinant). *frickeW_mem_GLPos : frickeW hN ∈ GLPos (Fin 2) ℝ.*

Theorem 3.2 (Involution). $W_N^2 = -N \cdot I$: *frickeW_sq_coe : ((frickeW hN * frickeW hN : GL (Fin 2) ℝ) : Matrix _ _ ℝ) = (-(N : ℝ)) • 1.*

Since scalar matrices act trivially on $\mathbb{H}$, Theorem 3.2 says precisely that W_N induces an involution of the upper half-plane.

The arithmetic input is the divisibility extracted from membership in $\Gamma_0(N)$, which Mathlib states modulo N :

The conjugate δ of (1) is then definable as a genuine $SL_2(\mathbb{Z})$ element, its determinant condition discharged by `linear_combination` against $\det \gamma = 1$:

```

theorem gamma0_dvd_lower_left {N : ℕ} {γ : SL(2, ℤ)} (hγ : γ ∈ Gamma0 N) :
  (N : ℤ) ∣ γ.1 1 0 := by
  rw [Gamma0_mem] at hγ
  exact (ZMod.intCast_zmod_eq_zero_iff_dvd _ _).mp hγ

```

Listing 2: Divisibility of the lower-left entry.

```

def frickeConj {N : ℕ} (γ : SL(2, ℤ)) (c' : ℤ) (hc : γ.1 1 0 = (N : ℤ) * c') : SL(2, ℤ) :=
  (!![γ.1 1 1, -c'; -(N : ℤ) * γ.1 0 1, γ.1 0 0], by
    have hdet := γ.2
    rw [Matrix.det_fin_two] at hdet; rw [hc] at hdet
    rw [Matrix.det_fin_two_of]; linear_combination hdet)

```

Listing 3: The Fricke conjugate, with its determinant proof.

Theorem 3.3 (Conjugation). *frickeW_conj_eq* : for $\gamma \in SL_2(\mathbb{Z})$ with $c = Nc'$,

$$W_N \cdot \text{mapGL}_{\mathbb{R}}(\gamma) \cdot W_N^{-1} = \text{mapGL}_{\mathbb{R}}(\text{frickeConj } \gamma c').$$

The proof rewrites by the intertwining identity (2) and cancels $W_N W_N^{-1}$.

Theorem 3.4 (Main result: normalization). *frickeW_normalizes_Gamma0* :

$$\forall g \in \text{Gamma0}(N). \text{map}(\text{mapGL}_{\mathbb{R}}), \quad W_N g W_N^{-1} \in \text{Gamma0}(N). \text{map}(\text{mapGL}_{\mathbb{R}}).$$

Proof (as formalized). Destructure $g = \text{mapGL}_{\mathbb{R}}(\gamma)$ with $\gamma \in \Gamma_0(N)$; obtain c' with $c = Nc'$ from `gamma0_dvd_lower_left`; rewrite by Theorem 3.3; conclude by `Subgroup.mem_map_of_mem` applied to `frickeConj_mem_Gamma0`, whose content is that the lower-left entry $-Nb$ vanishes modulo N . $\square$

4 The operator on modular forms

Normalization is a statement about groups; being an *operator on modular forms* requires carrying the two analytic side conditions of `ModularForm` along W_N . We do this in two steps, working with the image $\Gamma_0(N)_{\mathbb{R}} := \Gamma_0(N). \text{map}(\text{mapGL}_{\mathbb{R}})$ inside $GL_2(\mathbb{R})$, which is the subgroup Mathlib’s bundled types are stated over.

Theorem 4.1 (Slash-invariance). *slash_frickeW_invariant* : for f a *SlashInvariantForm* of weight k for $\Gamma_0(N)_{\mathbb{R}}$ and any g in that subgroup, $(f|_k W_N)|_k g = f|_k W_N$.

Proof (as formalized). Write $g = \text{mapGL}_{\mathbb{R}}(\gamma)$ with $\gamma \in \Gamma_0(N)$ and extract $c = Nc'$. Then, using the slash cocycle in both directions and the intertwining identity (2),

$$(f|_k W_N)|_k \gamma = f|_k (W_N \gamma) = f|_k (\delta W_N) = (f|_k \delta)|_k W_N = f|_k W_N,$$

the last step by slash-invariance of f under $\delta \in \Gamma_0(N)$ (Theorem 3.3). This packages as `frickeSlashOperator`. $\square$

Theorem 4.2 (Operator on `ModularForm`). *frickeModularOperator* : $f \mapsto f|_k W_N$ is a map `ModularForm` $\Gamma_0(N)_{\mathbb{R}} k \rightarrow \text{ModularForm } \Gamma_0(N)_{\mathbb{R}} k$.

Proof (as formalized). Holomorphy is immediate from Mathlib’s `MDifferentiable.slash`: slashing by any fixed element of $GL_2(\mathbb{R})$ preserves `MDiff`. For boundedness, Mathlib’s `OnePoint.IsBoundedAt.smul_`

reduces $f|_k W_N$ is bounded at c to f is bounded at $W_N \cdot c$, so it suffices that W_N carries cusps of $\Gamma_0(N)$ to cusps of $\Gamma_0(N)$. That is `isCusp_frickeW_smul`, obtained from Mathlib’s `IsCusp.smul` together with the subgroup form of Theorem 3.4 (`frickeW_conjAct_le`) and the monotonicity of `IsCusp` in its subgroup argument. $\square$

Concretely W_N exchanges the cusps 0 and ∞ of $\Gamma_0(N)$; what is proved here is the weaker statement, sufficient for the transport, that the cusp set is preserved.

Theorem 4.3 (The composite is a scalar). *frickeW_sq_slash : for every $f : \mathbb{H} \rightarrow \mathbb{C}$ and every weight k ,*

$$(f|_k W_N)|_k W_N = (N^2)^{k-1}(-N)^{-k} f = (-1)^k N^{k-2} f.$$

Proof (as formalized). By the cocycle law the left side is $f|_k(W_N^2)$, and $W_N^2 = -N \cdot I$ is scalar (Theorem 3.2). For that matrix the three ingredients of Mathlib’s slash are computed directly: $\det(W_N^2) = N^2$, $\text{denom}(W_N^2, \tau) = -N$, and W_N^2 acts trivially on $\mathbb{H}$. The conjugation factor σ needs no determinant case split: $\sigma(gg') = \sigma g \circ \sigma g'$ and $\sigma g \circ \sigma g = \text{id}$ give $\sigma(W_N^2) = \text{id}$ at once. $\square$

For even k it follows that $f \mapsto N^{1-k/2}(f|_k W_N)$ squares to the identity — the Fricke involution proper. For odd k the question is vacuous, since $-I \in \Gamma_0(N)$ forces $f \equiv 0$ (Mathlib’s `ModularForm.eq_zero_of_neg_one_mem`).

5 Verification status

Every declaration in the module compiles. There are no `sorry` placeholders and no project-local axioms. The axiom footprint of each theorem, obtained by `#print axioms`, is exactly

`[propext, Classical.choice, Quot.sound],`

the three standard axioms of Lean’s logic. Following the Fermat’s Last Theorem artifact [8], the footprint is not merely reported but *enforced*: a `FinalCheck` module places a `#guard_msgs in #print axioms` guard on each of the sixteen headline theorems inside the default build target, so any widening of the axiom set is a compile error, and a negative control (a guard asserting a deliberately wrong footprint) is kept outside the build and verified to fail. The development is tracked as a statement-dependency graph (`dag/theorems.jsonl`, with a human-readable map in `dag/PROOF-PATH.md`); the machine checks that every node marked proved names an existing declaration and depends only on proved nodes. We claim nothing beyond what the kernel checks: in the terminology we adopt throughout the SocrateAI programme, (C1)–(C7) are *formal* results; the surrounding exposition is not.

Artifact. Archived at Zenodo, [doi:10.5281/zenodo.22541930](https://doi.org/10.5281/zenodo.22541930), which is the citable version of record for this note and its sources. Repository <https://github.com/xaviercallens/SocrateAI-Lean-Lib> (tag `fricke-v1`); module `Lean/SocrateAI/ModularForms/{FrickeInvolution, FrickeSlash, FrickeModular, FrickeComposite}.lean`, with the axiom guards in `Lean/SocrateAI/FinalCheck.lean`; a mirror with a dataset card is at <https://huggingface.co/datasets/callensxavier/socrateai-fricke-inv>. Toolchain `leanprover/lean4:v4.32.2`; Mathlib pinned at commit `905b9581` (2026-07-28); lake build `SocrateAI` completes in 3053 jobs (including the axiom-guard module). The build configuration is *not* distributed: our `lakefile.lean` hard-codes absolute paths into a local Mathlib package pool, so the artifact is not yet buildable by a third party without supplying their own Mathlib at the pinned commit. A portable lakefile is outstanding work, and we flag it as the main obstacle to independent replication. The four modules total 27 declarations in 291 lines and import only `Mathlib.NumberTheory.ModularForms.SlashActions`, `Mathlib.NumberTheory.ModularForms.CongruenceSubgroups`, `Mathlib.NumberTheory.ModularForms.SlashI` and `Mathlib.NumberTheory.ModularForms.Basic`. A statement-dependency graph with a natural-language description per node, and a validator enforcing that every node marked proved names an existing declaration and depends only on proved nodes, is in `dag/`.

6 Related work

Prior Lean 4 formalization of Fricke material. The Fermat’s Last Theorem artifact [8] is a Lean 4 development built on Mathlib that already contains substantial Fricke and Atkin–Lehner theory. A repository-scoped code search returns on the order of 2.7×10^3 Lean files mentioning `Fricke` and 3.6×10^3 mentioning `AtkinLehner`, including `Thm_ModularForm_heckeU_add_slash_fricke_eq_zero`, `Thm_CuspForm_hasNebentypus_inv_and_qCoeff_hecke_eigen_of_fricke` and `Def_ModularCurve_AtkinLehner`. Their statements use the same matrix $\begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}$ and the same Mathlib slash action that we use here, and they go considerably further — to Hecke operators, eigenforms with nebentypus, and Atkin–Lehner operators on modular curves.

Most directly, `Def_CuspForm_AtkinLehnerOperator.lean` defines `atkinLehnerLin`, a map $\text{ModularForm}(\Gamma_0(M)) \ k \rightarrow_{\ell[\mathbb{C}]} \text{ModularForm}(\Gamma_0(M)) \ k$, discharging slash-invariance, holomorphy and boundedness at the cusps as our `frickeModularOperator` does — our (C5) and (C6), for the whole Atkin–Lehner family and as a $\mathbb{C}$ -linear map. Their `Def_ModularForm_AtkinLehnerDatum.lean` builds its group element with the same `mkOfDetNeZero` idiom we use and proves the analogue of our (C1), (C2) and (C7).

In particular their `CohCarrier.frickeMat` is, up to naming, our `frickeConj`: the same conjugate matrix $\begin{pmatrix} d & -c/N \\ -Nb & a \end{pmatrix}$, the same divisibility hypothesis, and a determinant proof that likewise finishes with `linear_combination`. Our development was arrived at independently, which does not bear on priority; we record the coincidence because a reader comparing the two should know they are the same construction.

What is nonetheless offered here. The FLT material is proof-internal: it lives in a bespoke `CohCarrier` namespace, and W_N is generally passed to each theorem as a parameter constrained by a matrix equation. We instead define W_N once as a `GL (Fin 2) ℝ` element and prove the small surrounding theory — membership in GL_2^+ , $W_N^2 = -N \cdot I$, the conjugation formula, and the induced operators on `SlashInvariantForm` and `ModularForm`. The intended destination is Mathlib, which has none of this. Stated precisely, what is left after disclosure is: the literal matrix W_N over $\mathbb{R}$ (their datum’s lower-right entry is q , so it does not specialize to W_N on the nose), the explicit composite scalar of (C7), and a four-named-import development using stock tactics rather than a bespoke proof harness. That is a methodology difference, and whether it is worth anything is a question for Mathlib review, not for us to assert.

7 Limitations

Fricke and Atkin–Lehner operators are classical; the definitive treatment of the Atkin–Lehner involutions W_Q for $Q \parallel N$ is [1], and textbook accounts appear in [4, 5, 6]. Our contribution is not mathematical novelty but the transfer of this material into a machine-checked setting where it can be depended upon.

We state the limitations plainly.

Remark 7.1 (What is *not* done here).

- (a) We formalize the Fricke involution W_N only. The full Atkin–Lehner family W_Q for $Q \parallel N$, and the resulting simultaneous eigenspace decomposition, are not treated.
- (b) The ± 1 eigenspace decomposition of $M_k(\Gamma_0(N))$ is not itself packaged as a Lean definition. The mathematical content is Theorem 4.3, which is formalized; what remains is bookkeeping — carrying the even- k hypothesis through the `zpow` arithmetic on the constant and bundling the two eigenspaces as submodules.

- (c) We make no priority claim. Our gap check was run against Mathlib, where it holds; a later search of the Lean 4 developments cited here found the prior art described in §6. We have not searched the Lean Zulip or other proof assistants.

8 Intended application

The motivation is downstream. Eta-quotients $f(\tau) = \prod_{\delta|N} \eta(\delta\tau)^{r_\delta}$ are modular on $\Gamma_0(N)$ under Ligozat’s conditions, and their behaviour under Fricke and Atkin–Lehner operators is what makes multiplicativity tractable [7]. A formalization of eta-quotient identities therefore needs W_N as a prerequisite, which is what this module supplies. The same operator is the modular-side input to the moonshine correspondences studied elsewhere in this programme. With the operator now defined on `ModularForm`, the immediate next steps are the normalization constant of limitation (b) and the full Atkin–Lehner family of limitation (a). On the analytic side, the Rademacher-type coefficient bounds that accompany the eta-quotient programme will first be checked at the program-verified tier with lightweight inequality-proof tooling in the style of Tao’s *estimates* assistant [11], and only then promoted to kernel-checked statements — the same two-tier discipline (exact checker below, kernel above) used throughout this programme.

9 Conclusion

We have formalized the Fricke involution W_N on $\Gamma_0(N)$ in Lean 4 and the operator it induces on modular forms, addressing a gap in *Mathlib* — though not, as §6 records, in Lean 4 as a whole. The development is small — 27 declarations in 291 lines — depends only on Lean’s three standard axioms, and is built on Mathlib’s existing `SlashAction`, `SlashInvariantForm` and `ModularForm` infrastructure rather than duplicating it, so that it composes with the rest of the library. The mathematics is classical; the machine-checked statement of it is not.

References

- [1] A. O. L. Atkin, J. Lehner, *Hecke operators on $\Gamma_0(m)$* , Math. Ann. **185** (1970), 134–160. doi:10.1007/BF01359701.
- [2] The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP), 2020.
- [3] K. Buzzard, *Formalising mathematics*, Math. Proc. Cambridge Philos. Soc., 2021.
- [4] F. Diamond, J. Shurman, *A First Course in Modular Forms*, Graduate Texts in Mathematics **228**, Springer (2005).
- [5] T. Miyake, *Modular Forms*, Springer Monographs in Mathematics (2006).
- [6] K. Ono, *The Web of Modularity: Arithmetic of the Coefficients of Modular Forms and q -series*, CBMS Regional Conference Series in Mathematics **102**, AMS (2004). doi:10.1090/cbms/102.
- [7] Y. Martin, *Multiplicative eta-quotients*, Trans. Amer. Math. Soc. **348** (1996), 4825–4856.
- [8] Anthropic, *A formal proof of Fermat’s Last Theorem in Lean 4*, software artifact, <https://github.com/anthropics/fermats-last-theorem>, 2026.
- [9] S. Chen, K. Marwaha, X. Lu, H. Yuen, T. Peng, *Prove2Me: An open collaborative platform for scaling math formalization*, arXiv:2608.28433, 2026.

- [10] T. Tao, *Mathematics in the age of AI*, lecture at the International Congress of Mathematicians 2026; arXiv:2608.16753.
- [11] T. Tao, *estimates: a lightweight proof assistant for inequalities and asymptotic estimates*, software, <https://github.com/teorth/estimates>, 2025.
- [12] L. de Moura et al., *The Lean 4 Theorem Prover and Programming Language*, CADE-28, LNCS **12699**, Springer (2021).